Student Name :Vidya Rajan Dharne

Batch :CS01

Roll Number :CS12

Aim

To implement graph-based clustering using Minimum Spanning Tree (MST) on the Breast Cancer Wisconsin (Diagnostic) dataset in Python and visualize the resulting clusters.

Objectives

1. To understand the concept of graph-based clustering and Minimum Spanning Trees (MST).
2. To load and preprocess the Breast Cancer Wisconsin dataset (scaling features and handling metadata).
3. To construct a distance graph from tumor feature representations using Euclidean distance.
4. To compute the Minimum Spanning Tree using Prim's or Kruskal's algorithm (or via scipy/networkx).
5. To form $K = 2$ clusters (Malignant vs. Benign) by removing the single $(K - 1)$ largest edge weight from the MST.
6. To evaluate the performance of the MST-based clustering model using evaluation metrics.

Relevant Theory

###Graph-Based Clustering & Minimum Spanning Tree (MST) Graph-based clustering represents data points as nodes in a graph, with edges representing relationships or distance metrics between individual samples.

- **Complete Graph Construction**: Given a dataset $X = \{x_1, x_2, \ldots, x_n\}$, each tissue sample is treated as a vertex. An edge weight $w(i, j)$ corresponds to the distance between sample i and sample j (typically Euclidean distance on scaled features):
$$w(i, j) = \|x_i - x_j\|_2$$
- **Minimum Spanning Tree (MST)**: An MST is a subset of edges connecting all vertices in the graph without forming any cycles, while minimizing the total edge weight sum.
- **MST-Based Clustering Method**:

    1. Construct a fully connected graph with pairwise edge weights across standardized features.

    2. Compute the Minimum Spanning Tree (MST).

3. To partition the data into K distinct clusters (such as Malignant and Benign groups), identify and remove the $K - 1$ largest (most expensive) edges from the MST.4. The resulting disconnected subgraphs represent the separate data clusters.

Dataset Information

Get the Breast Cancer Wisconsin (Diagnostic) dataset from Kaggle or via scikit-learn (load_breast_cancer). The dataset contains key cell nuclei features computed from digitized images of fine needle aspirates (FNA) of breast masses:

- Total Instances: 569 samples
- Target Classes: Malignant (M) and Benign (B)* Feature Attributes: 30 numerical features (e.g., mean radius, mean texture, mean perimeter, mean area, mean smoothness)

For 2D visualization, two key features (such as mean radius and mean texture) will be selected.

Task 1: Import Libraries and Load Dataset

- Import required libraries (numpy, pandas, matplotlib, scipy, networkx, sklearn).
- Load the Breast Cancer dataset using below link from Kaggle.

  https://www.kaggle.com/datasets/utkarshx27/breast-cancer-wisconsin-diagnostic-dataset

---

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy
import networkx as nx

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix,
        classification_report

# Load Breast Cancer Wisconsin dataset
df = pd.read_csv("brca.csv")

# Display first 5 rows
print("First 5 rows of the dataset:")
print(df.head())

# Display dataset shape
print("\nDataset Shape:")
print(df.shape)
```

```
# Display column names
print("\nColumn Names:")
print(df.columns.tolist())
```

```
First 5 rows of the dataset:
   Unnamed: 0  x.radius_mean  x.texture_mean  x.perimeter_mean  x.area_mean
\
0           1         13.540           14.36             87.46        566.3
1           2         13.080           15.71             85.63        520.0
2           3          9.504           12.44             60.34        273.9
3           4         13.030           18.42             82.61        523.8
4           5          8.196           16.84             51.71        201.9

   x.smoothness_mean  x.compactness_mean  x.concavity_mean  \
0            0.09779             0.08129           0.06664
1            0.10750             0.12700           0.04568
2            0.10240             0.06492           0.02956
3            0.08983             0.03766           0.02562
4            0.08600             0.05943           0.01588

   x.concave_pts_mean  x.symmetry_mean  ...  x.texture_worst  \
0            0.047810           0.1885  ...            19.26
1            0.031100           0.1967  ...            20.49
2            0.020760           0.1815  ...            15.66
3            0.029230           0.1467  ...            22.81
4            0.005917           0.1769  ...            21.96

   x.perimeter_worst  x.area_worst  x.smoothness_worst  x.compactness_worst
\
0              99.70         711.2             0.14400              0.17730
1              96.09         630.5             0.13120              0.27760
2              65.13         314.9             0.13240              0.11480
3              84.46         545.9             0.09701              0.04619
4              57.26         242.2             0.12970              0.13570

   x.concavity_worst  x.concave_pts_worst  x.symmetry_worst  \
0            0.23900              0.12880            0.2977
1            0.18900              0.07283            0.3184
2            0.08867              0.06227            0.2450
3            0.04833              0.05013            0.1987
4            0.06880              0.02564            0.3105

   x.fractal_dim_worst  y
0              0.07259  B
1              0.08183  B
2              0.07773  B
3              0.06169  B
4              0.07409  B

[5 rows x 32 columns]

Dataset Shape:
(569, 32)

Column Names:
['Unnamed: 0', 'x.radius_mean', 'x.texture_mean', 'x.perimeter_mean',
'x.area_mean', 'x.smoothness_mean', 'x.compactness_mean', 'x.concavity_mean',
'x.concave_pts_mean', 'x.symmetry_mean', 'x.fractal_dim_mean', 'x.radius_se',
'x.texture_se', 'x.perimeter_se', 'x.area_se', 'x.smoothness_se',
'x.compactness_se', 'x.concavity_se', 'x.concave_pts_se', 'x.symmetry_se',
'x.fractal_dim_se', 'x.radius_worst', 'x.texture_worst', 'x.perimeter_worst',
'x.area_worst', 'x.smoothness_worst', 'x.compactness_worst',
'x.concavity_worst', 'x.concave_pts_worst', 'x.symmetry_worst',
'x.fractal_dim_worst', 'y']
```

Task 2: Explore the Dataset

- Display missing value checks, summary statistics, and feature dimensions.
- Check the diagnosis class distribution (Malignant vs. Benign).

```
# Task 2: Explore the Dataset

# Display column names
print("Column Names:")
print(df.columns.tolist())

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Summary statistics
print("\nSummary Statistics:")
print(df.describe())

# Dataset dimensions
print("\nDataset Dimensions:")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])

# Find diagnosis column automatically
diagnosis_col = None

for col in df.columns:
    if col.strip().lower() in ["diagnosis", "diagnosis "]:
        diagnosis_col = col
        break

# Check class distribution
if diagnosis_col:
    print("\nDiagnosis Class Distribution:")
    print(df[diagnosis_col].value_counts())

    print("\nMalignant (M):", (df[diagnosis_col] == "M").sum())
    print("Benign (B):", (df[diagnosis_col] == "B").sum())
else:
    print("\nDiagnosis column not found.")
```

```
Column Names:
['Unnamed: 0', 'x.radius_mean', 'x.texture_mean', 'x.perimeter_mean',
'x.area_mean', 'x.smoothness_mean', 'x.compactness_mean', 'x.concavity_mean',
'x.concave_pts_mean', 'x.symmetry_mean', 'x.fractal_dim_mean', 'x.radius_se',
'x.texture_se', 'x.perimeter_se', 'x.area_se', 'x.smoothness_se',
'x.compactness_se', 'x.concavity_se', 'x.concave_pts_se', 'x.symmetry_se',
'x.fractal_dim_se', 'x.radius_worst', 'x.texture_worst', 'x.perimeter_worst',
'x.area_worst', 'x.smoothness_worst', 'x.compactness_worst',
'x.concavity_worst', 'x.concave_pts_worst', 'x.symmetry_worst',
'x.fractal_dim_worst', 'y']

Missing Values:
Unnamed: 0             0
x.radius_mean          0
x.texture_mean         0
x.perimeter_mean       0
```

```
x.area_mean            0
x.smoothness_mean      0
x.compactness_mean     0
x.concavity_mean       0
x.concave_pts_mean     0
x.symmetry_mean        0
x.fractal_dim_mean     0
x.radius_se            0
x.texture_se           0
x.perimeter_se         0
x.area_se              0
x.smoothness_se        0
x.compactness_se       0
x.concavity_se         0
x.concave_pts_se       0
x.symmetry_se          0
x.fractal_dim_se       0
x.radius_worst         0
x.texture_worst        0
x.perimeter_worst      0
x.area_worst           0
x.smoothness_worst     0
x.compactness_worst    0
x.concavity_worst      0
x.concave_pts_worst    0
x.symmetry_worst       0
x.fractal_dim_worst    0
y                      0
dtype: int64

Summary Statistics:
       Unnamed: 0  x.radius_mean  x.texture_mean  x.perimeter_mean  \
count  569.000000     569.000000      569.000000        569.000000
mean   285.000000      14.127292       19.289649         91.969033
std    164.400426       3.524049        4.301036         24.298981
min      1.000000       6.981000        9.710000         43.790000
25%    143.000000      11.700000       16.170000         75.170000
50%    285.000000      13.370000       18.840000         86.240000
75%    427.000000      15.780000       21.800000        104.100000
max    569.000000      28.110000       39.280000        188.500000

       x.area_mean  x.smoothness_mean  x.compactness_mean  x.concavity_mean
\
count   569.000000         569.000000          569.000000        569.000000
mean    654.889104           0.096360            0.104341          0.088799
std     351.914129           0.014064            0.052813          0.079720
min     143.500000           0.052630            0.019380          0.000000
25%     420.300000           0.086370            0.064920          0.029560
50%     551.100000           0.095870            0.092630          0.061540
75%     782.700000           0.105300            0.130400          0.130700
max    2501.000000           0.163400            0.345400          0.426800

       x.concave_pts_mean  x.symmetry_mean  ...  x.radius_worst  \
count          569.000000       569.000000  ...      569.000000
mean             0.048919         0.181162  ...       16.269190
std              0.038803         0.027414  ...        4.833242
min              0.000000         0.106000  ...        7.930000
25%              0.020310         0.161900  ...       13.010000
50%              0.033500         0.179200  ...       14.970000
75%              0.074000         0.195700  ...       18.790000
max              0.201200         0.304000  ...       36.040000

       x.texture_worst  x.perimeter_worst  x.area_worst  x.smoothness_worst
\
count       569.000000         569.000000    569.000000          569.000000
```

```
mean         25.677223         107.261213    880.583128            0.132369
std           6.146258          33.602542    569.356993            0.022832
min          12.020000          50.410000    185.200000            0.071170
25%          21.080000          84.110000    515.300000            0.116600
50%          25.410000          97.660000    686.500000            0.131300
75%          29.720000         125.400000   1084.000000            0.146000
max          49.540000         251.200000   4254.000000            0.222600

       x.compactness_worst  x.concavity_worst  x.concave_pts_worst  \
count           569.000000         569.000000           569.000000
mean              0.254265           0.272188             0.114606
std               0.157336           0.208624             0.065732
min               0.027290           0.000000             0.000000
25%               0.147200           0.114500             0.064930
50%               0.211900           0.226700             0.099930
75%               0.339100           0.382900             0.161400
max               1.058000           1.252000             0.291000

       x.symmetry_worst  x.fractal_dim_worst
count        569.000000           569.000000
mean           0.290076             0.083946
std            0.061867             0.018061
min            0.156500             0.055040
25%            0.250400             0.071460
50%            0.282200             0.080040
75%            0.317900             0.092080
max            0.663800             0.207500

[8 rows x 31 columns]

Dataset Dimensions:
Rows: 569
Columns: 32

Diagnosis column not found.
```

Task 3: Preprocess and Scale Features

- Select numerical features and drop non-informative identifiers (such as id).
- Apply StandardScaler to normalize feature magnitudes prior to graph distance calculation.

```python
# Task 3: Preprocess and Scale Features

from sklearn.preprocessing import StandardScaler

# Select only numerical columns
X = df.select_dtypes(include=["int64", "float64"])

# Remove ID column if present
if "id" in X.columns:
    X = X.drop(columns=["id"])

# Apply StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Convert scaled data into DataFrame
```

```
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

# Display information
print("Original Feature Shape:", X.shape)
print("Scaled Feature Shape:", X_scaled.shape)

print("\nFirst 5 rows after scaling:")
print(X_scaled.head())
```

```
Original Feature Shape: (569, 31)
Scaled Feature Shape: (569, 31)

First 5 rows after scaling:
   Unnamed: 0  x.radius_mean  x.texture_mean  x.perimeter_mean  x.area_mean
\
0   -1.729009      -0.166799       -1.147162         -0.185728    -0.251957
1   -1.722921      -0.297446       -0.833008         -0.261106    -0.383638
2   -1.716833      -1.313080       -1.593959         -1.302806    -1.083572
3   -1.710745      -0.311646       -0.202373         -0.385500    -0.372831
4   -1.704657      -1.684571       -0.570050         -1.658278    -1.288347

   x.smoothness_mean  x.compactness_mean  x.concavity_mean  \
0           0.101747           -0.436850         -0.278210
1           0.792763            0.429422         -0.541362
2           0.429819           -0.747086         -0.743748
3          -0.464730           -1.263703         -0.793214
4          -0.737294           -0.851130         -0.915500

   x.concave_pts_mean  x.symmetry_mean  ...  x.radius_worst  x.texture_worst
\
0           -0.028609         0.267911  ...       -0.240048        -1.045005
1           -0.459627         0.567289  ...       -0.366368        -0.844707
2           -0.726337         0.012345  ...       -1.250611        -1.631243
3           -0.507861        -1.258183  ...       -0.614867        -0.466909
4           -1.109197        -0.155598  ...       -1.512777        -0.605327

   x.perimeter_worst  x.area_worst  x.smoothness_worst  x.compactness_worst
\
0          -0.225217     -0.297761            0.509873            -0.489605
1          -0.332744     -0.439624           -0.051226             0.148443
2          -1.254913     -0.994422            0.001377            -0.887193
3          -0.679153     -0.588344           -1.549975            -1.323648
4          -1.489328     -1.122222           -0.116980            -0.754239

   x.concavity_worst  x.concave_pts_worst  x.symmetry_worst  \
0          -0.159223             0.216123          0.123347
1          -0.399099            -0.636110          0.458227
2          -0.880434            -0.796903         -0.729224
3          -1.073966            -0.981753         -1.478256
4          -0.975761            -1.354653          0.330422

   x.fractal_dim_worst
0            -0.629292
1            -0.117250
2            -0.344455
3            -1.233324
4            -0.546168

[5 rows x 31 columns]
```

Task 4: Construct Pairwise Distance Matrix & Graph

- Compute pairwise Euclidean distance matrix across all scaled samples using scipy.spatial.distance.pdist or cdist.
- Represent the dataset as a weighted complete graph.

```
# Task 4: Construct Pairwise Distance Matrix & Graph

import numpy as np
import matplotlib.pyplot as plt
import networkx as nx
from scipy.spatial.distance import pdist, squareform

# Compute pairwise Euclidean distances
distance_vector = pdist(X_scaled, metric="euclidean")

# Convert distances into a square matrix
distance_matrix = squareform(distance_vector)

print("Distance Matrix Shape:")
print(distance_matrix.shape)

print("\nFirst 5 x 5 Distance Matrix:")
print(distance_matrix[:5, :5])

# Create weighted complete graph using first 20 samples
n = 20

G = nx.Graph()

# Add nodes
for i in range(n):
    G.add_node(i)

# Add weighted edges
for i in range(n):
    for j in range(i + 1, n):
        G.add_edge(i, j, weight=distance_matrix[i, j])

print("\nNumber of Nodes:", G.number_of_nodes())
print("Number of Edges:", G.number_of_edges())
```

```
Distance Matrix Shape:
(569, 569)

First 5 x 5 Distance Matrix:
[[0.         3.10725264 3.62316224 5.20451369 4.46715199]
 [3.10725264 0.         4.01028047 5.62303335 4.37102093]
 [3.62316224 4.01028047 0.         5.12193953 2.94842089]
 [5.20451369 5.62303335 5.12193953 0.         5.02799161]
 [4.46715199 4.37102093 2.94842089 5.02799161 0.        ]]

Number of Nodes: 20
Number of Edges: 190
```

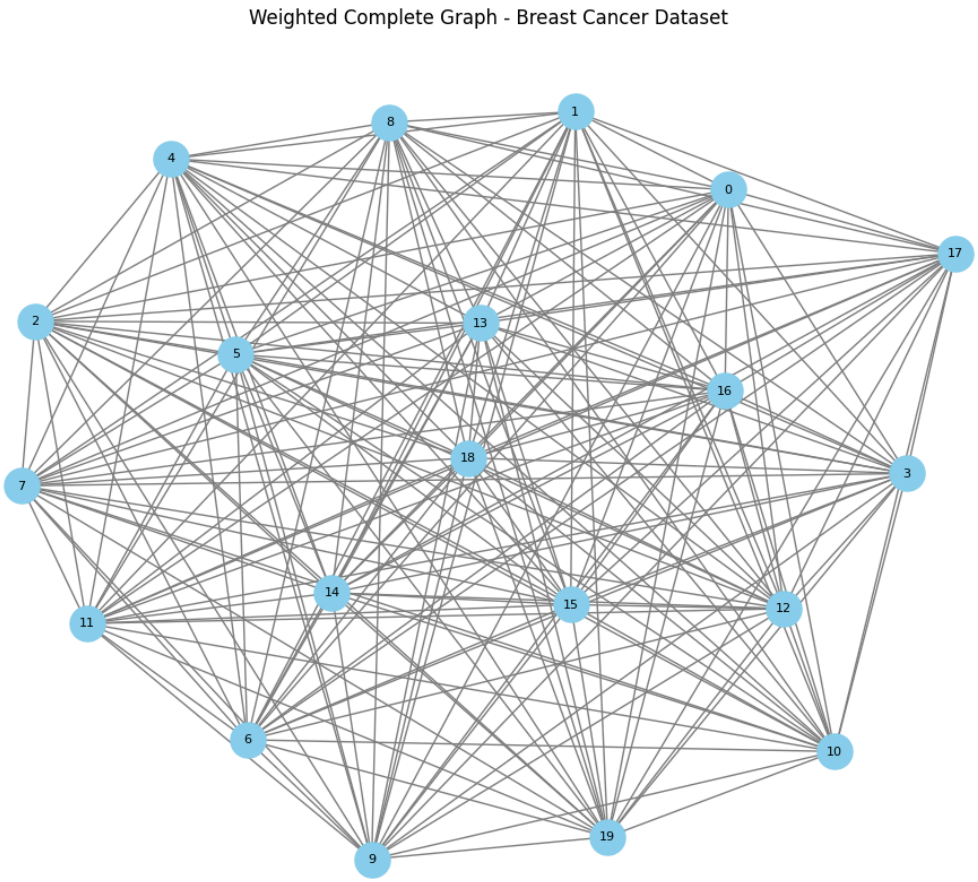
```
# Visualize the weighted complete graph

plt.figure(figsize=(10, 8))

pos = nx.spring_layout(G, seed=42)

nx.draw(
    G,
    pos,
    with_labels=True,
    node_size=500,
    node_color="skyblue",
    font_size=8,
    edge_color="gray"
)

plt.title("Weighted Complete Graph - Breast Cancer Dataset")
plt.show()
```



Task 5: Compute the Minimum Spanning Tree (MST)

- Compute the MST using scipy.sparse.csgraph.minimum_spanning_tree or networkx.minimum_spanning_tree.

- Extract the resulting graph connections and weights.

```
# Task 5: Compute Minimum Spanning Tree (MST)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import networkx as nx
from scipy.sparse.csgraph import minimum_spanning_tree

# Compute MST from the distance matrix
mst_sparse = minimum_spanning_tree(distance_matrix)

# Convert MST to a normal NumPy matrix
mst_matrix = mst_sparse.toarray()

# Display MST matrix
print("MST Matrix:")
print(mst_matrix)

# Create NetworkX graph from MST matrix
MST = nx.from_numpy_array(mst_matrix)

# Remove zero-weight edges if any
MST.remove_edges_from(
    [(u, v) for u, v, d in MST.edges(data=True) if d["weight"] == 0]
)

# Display MST connections and weights
print("\nMST Connections and Weights:")
for u, v, data in MST.edges(data=True):
    print(f"Node {u} -- Node {v} : Weight = {data['weight']:.3f}")

print("\nNumber of MST Nodes:", MST.number_of_nodes())
print("Number of MST Edges:", MST.number_of_edges())

# ----------------------------
# MST Graph Visualization
# ----------------------------

plt.figure(figsize=(12, 8))

pos = nx.spring_layout(MST, seed=42)

nx.draw(
    MST,
    pos,
    with_labels=True,
    node_size=500,
    node_color="skyblue",
    font_size=8,
    edge_color="gray"
)

# Display edge weights
edge_labels = nx.get_edge_attributes(MST, "weight")

nx.draw_networkx_edge_labels(
    MST,
```

```
    pos,
    edge_labels=edge_labels,
    font_size=7
)

plt.title("Minimum Spanning Tree (MST)")
plt.show()

# ----------------------------
# MST Matrix Visualization
# ----------------------------

plt.figure(figsize=(8, 6))

plt.imshow(mst_matrix, cmap="viridis", aspect="auto")
plt.colorbar(label="Edge Weight")

plt.title("MST Adjacency Matrix")
plt.xlabel("Sample Index")
plt.ylabel("Sample Index")

plt.show()

MST Matrix:
[[0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 ...
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]]

MST Connections and Weights:
Node 0 -- Node 81 : Weight = 1.759
Node 0 -- Node 108 : Weight = 2.269
Node 1 -- Node 105 : Weight = 1.777
Node 2 -- Node 58 : Weight = 3.023
Node 2 -- Node 70 : Weight = 2.257
Node 3 -- Node 170 : Weight = 2.557
Node 4 -- Node 84 : Weight = 2.220
Node 4 -- Node 114 : Weight = 2.358
Node 5 -- Node 34 : Weight = 1.824
Node 6 -- Node 29 : Weight = 2.051
Node 7 -- Node 140 : Weight = 1.949
Node 8 -- Node 66 : Weight = 1.552
Node 9 -- Node 95 : Weight = 1.753
Node 9 -- Node 126 : Weight = 1.729
Node 10 -- Node 126 : Weight = 1.913
Node 11 -- Node 140 : Weight = 1.540
Node 12 -- Node 322 : Weight = 2.747
Node 13 -- Node 191 : Weight = 3.376
Node 13 -- Node 247 : Weight = 3.545
Node 14 -- Node 16 : Weight = 2.746
Node 14 -- Node 121 : Weight = 3.029
Node 15 -- Node 103 : Weight = 3.671
Node 16 -- Node 24 : Weight = 1.769
Node 17 -- Node 19 : Weight = 1.831
Node 18 -- Node 69 : Weight = 10.481
Node 18 -- Node 208 : Weight = 7.876
Node 19 -- Node 90 : Weight = 1.635
Node 19 -- Node 117 : Weight = 1.679
Node 19 -- Node 174 : Weight = 1.653
Node 20 -- Node 85 : Weight = 5.272
```

Node 21 -- Node 23 : Weight = 1.231
Node 21 -- Node 40 : Weight = 1.727
Node 21 -- Node 57 : Weight = 1.169
Node 21 -- Node 101 : Weight = 1.438
Node 22 -- Node 186 : Weight = 3.661
Node 23 -- Node 31 : Weight = 1.352
Node 23 -- Node 97 : Weight = 1.910
Node 23 -- Node 201 : Weight = 1.479
Node 24 -- Node 39 : Weight = 2.427
Node 24 -- Node 77 : Weight = 2.434
Node 25 -- Node 303 : Weight = 2.869
Node 26 -- Node 72 : Weight = 1.540
Node 26 -- Node 129 : Weight = 1.519
Node 27 -- Node 96 : Weight = 1.936
Node 28 -- Node 53 : Weight = 2.444
Node 28 -- Node 183 : Weight = 2.609
Node 29 -- Node 120 : Weight = 2.081
Node 29 -- Node 200 : Weight = 2.061
Node 30 -- Node 66 : Weight = 2.198
Node 30 -- Node 299 : Weight = 3.333
Node 31 -- Node 80 : Weight = 1.681
Node 32 -- Node 60 : Weight = 2.521
Node 33 -- Node 155 : Weight = 2.523
Node 34 -- Node 49 : Weight = 1.345
Node 34 -- Node 104 : Weight = 2.116
Node 34 -- Node 132 : Weight = 1.406
Node 35 -- Node 106 : Weight = 3.305
Node 36 -- Node 200 : Weight = 1.962
Node 37 -- Node 39 : Weight = 2.135
Node 37 -- Node 96 : Weight = 2.246
Node 38 -- Node 67 : Weight = 2.775
Node 38 -- Node 106 : Weight = 2.240
Node 41 -- Node 47 : Weight = 2.167
Node 42 -- Node 82 : Weight = 2.833
Node 42 -- Node 171 : Weight = 2.805
Node 42 -- Node 187 : Weight = 3.021
Node 43 -- Node 45 : Weight = 2.763
Node 43 -- Node 48 : Weight = 4.821
Node 43 -- Node 143 : Weight = 5.657
Node 44 -- Node 208 : Weight = 4.525
Node 45 -- Node 171 : Weight = 2.252
Node 46 -- Node 322 : Weight = 2.357
Node 47 -- Node 101 : Weight = 1.956
Node 50 -- Node 105 : Weight = 1.911
Node 51 -- Node 224 : Weight = 2.322
Node 52 -- Node 66 : Weight = 1.586
Node 52 -- Node 136 : Weight = 1.571
Node 52 -- Node 203 : Weight = 1.619
Node 53 -- Node 244 : Weight = 3.268
Node 54 -- Node 147 : Weight = 1.895
Node 55 -- Node 65 : Weight = 1.761
Node 55 -- Node 232 : Weight = 1.667
Node 56 -- Node 231 : Weight = 2.876
Node 58 -- Node 134 : Weight = 3.723
Node 59 -- Node 84 : Weight = 2.019
Node 60 -- Node 155 : Weight = 2.089
Node 60 -- Node 182 : Weight = 1.859
Node 61 -- Node 204 : Weight = 1.648
Node 62 -- Node 75 : Weight = 1.859
Node 62 -- Node 92 : Weight = 1.828
Node 63 -- Node 171 : Weight = 3.008
Node 64 -- Node 233 : Weight = 2.670
Node 65 -- Node 183 : Weight = 1.970
Node 65 -- Node 294 : Weight = 2.752
Node 66 -- Node 152 : Weight = 1.636

Node 68 -- Node 119 : Weight = 3.929
Node 68 -- Node 333 : Weight = 4.343
Node 70 -- Node 81 : Weight = 1.717
Node 70 -- Node 135 : Weight = 1.868
Node 71 -- Node 147 : Weight = 1.617
Node 72 -- Node 95 : Weight = 1.614
Node 72 -- Node 148 : Weight = 1.679
Node 72 -- Node 212 : Weight = 1.806
Node 73 -- Node 128 : Weight = 2.929
Node 74 -- Node 79 : Weight = 2.329
Node 74 -- Node 132 : Weight = 1.573
Node 74 -- Node 149 : Weight = 1.301
Node 74 -- Node 173 : Weight = 1.467
Node 74 -- Node 227 : Weight = 1.686
Node 76 -- Node 127 : Weight = 2.242
Node 76 -- Node 195 : Weight = 2.572
Node 78 -- Node 136 : Weight = 1.487
Node 81 -- Node 132 : Weight = 1.677
Node 83 -- Node 135 : Weight = 2.091
Node 84 -- Node 159 : Weight = 2.424
Node 85 -- Node 145 : Weight = 5.309
Node 85 -- Node 169 : Weight = 5.431
Node 86 -- Node 131 : Weight = 2.295
Node 87 -- Node 175 : Weight = 1.895
Node 88 -- Node 139 : Weight = 2.502
Node 89 -- Node 279 : Weight = 2.281
Node 91 -- Node 178 : Weight = 2.715
Node 91 -- Node 242 : Weight = 1.967
Node 92 -- Node 95 : Weight = 1.576
Node 93 -- Node 233 : Weight = 3.395
Node 94 -- Node 284 : Weight = 5.609
Node 95 -- Node 137 : Weight = 1.999
Node 95 -- Node 206 : Weight = 1.588
Node 96 -- Node 97 : Weight = 1.682
Node 97 -- Node 320 : Weight = 2.077
Node 98 -- Node 106 : Weight = 1.601
Node 98 -- Node 133 : Weight = 2.393
Node 98 -- Node 182 : Weight = 1.809
Node 99 -- Node 177 : Weight = 2.808
Node 100 -- Node 205 : Weight = 2.143
Node 101 -- Node 107 : Weight = 1.609
Node 101 -- Node 156 : Weight = 1.636
Node 101 -- Node 173 : Weight = 1.483
Node 101 -- Node 188 : Weight = 1.610
Node 101 -- Node 290 : Weight = 1.684
Node 102 -- Node 195 : Weight = 1.871
Node 103 -- Node 228 : Weight = 2.720
Node 105 -- Node 245 : Weight = 2.090
Node 105 -- Node 292 : Weight = 2.063
Node 107 -- Node 276 : Weight = 1.780
Node 109 -- Node 132 : Weight = 1.813
Node 110 -- Node 183 : Weight = 1.887
Node 111 -- Node 278 : Weight = 2.690
Node 112 -- Node 209 : Weight = 2.646
Node 113 -- Node 273 : Weight = 2.702
Node 114 -- Node 190 : Weight = 1.908
Node 115 -- Node 257 : Weight = 2.010
Node 116 -- Node 269 : Weight = 2.895
Node 117 -- Node 254 : Weight = 1.603
Node 118 -- Node 158 : Weight = 1.714
Node 118 -- Node 168 : Weight = 1.762
Node 119 -- Node 277 : Weight = 3.949
Node 121 -- Node 240 : Weight = 2.029
Node 122 -- Node 222 : Weight = 2.007
Node 123 -- Node 277 : Weight = 2.687

Node 124 -- Node 250 : Weight = 2.371
Node 125 -- Node 132 : Weight = 1.545
Node 125 -- Node 172 : Weight = 2.258
Node 125 -- Node 245 : Weight = 1.768
Node 125 -- Node 289 : Weight = 2.037
Node 127 -- Node 154 : Weight = 2.168
Node 128 -- Node 200 : Weight = 1.736
Node 130 -- Node 171 : Weight = 2.166
Node 131 -- Node 160 : Weight = 1.638
Node 131 -- Node 217 : Weight = 1.685
Node 132 -- Node 190 : Weight = 2.014
Node 132 -- Node 219 : Weight = 1.218
Node 132 -- Node 241 : Weight = 1.860
Node 132 -- Node 326 : Weight = 1.965
Node 133 -- Node 220 : Weight = 3.393
Node 134 -- Node 325 : Weight = 3.931
Node 136 -- Node 163 : Weight = 1.515
Node 136 -- Node 196 : Weight = 1.627
Node 136 -- Node 291 : Weight = 1.739
Node 138 -- Node 197 : Weight = 2.995
Node 139 -- Node 194 : Weight = 2.281
Node 139 -- Node 224 : Weight = 1.963
Node 140 -- Node 192 : Weight = 1.965
Node 140 -- Node 344 : Weight = 2.318
Node 141 -- Node 218 : Weight = 3.909
Node 141 -- Node 265 : Weight = 2.576
Node 142 -- Node 160 : Weight = 1.666
Node 144 -- Node 157 : Weight = 2.212
Node 144 -- Node 162 : Weight = 2.021
Node 144 -- Node 234 : Weight = 2.024
Node 144 -- Node 261 : Weight = 3.557
Node 146 -- Node 246 : Weight = 2.022
Node 147 -- Node 185 : Weight = 1.634
Node 148 -- Node 162 : Weight = 1.771
Node 149 -- Node 150 : Weight = 1.744
Node 151 -- Node 251 : Weight = 2.298
Node 152 -- Node 288 : Weight = 1.415
Node 153 -- Node 242 : Weight = 2.842
Node 153 -- Node 347 : Weight = 2.496
Node 154 -- Node 193 : Weight = 2.772
Node 154 -- Node 224 : Weight = 2.253
Node 154 -- Node 228 : Weight = 2.263
Node 154 -- Node 233 : Weight = 2.570
Node 155 -- Node 231 : Weight = 1.994
Node 158 -- Node 203 : Weight = 1.578
Node 160 -- Node 161 : Weight = 1.490
Node 161 -- Node 260 : Weight = 1.742
Node 164 -- Node 216 : Weight = 1.700
Node 164 -- Node 241 : Weight = 1.872
Node 165 -- Node 219 : Weight = 2.227
Node 166 -- Node 178 : Weight = 5.139
Node 167 -- Node 176 : Weight = 1.493
Node 168 -- Node 192 : Weight = 1.731
Node 169 -- Node 295 : Weight = 3.871
Node 170 -- Node 300 : Weight = 2.778
Node 171 -- Node 253 : Weight = 2.171
Node 171 -- Node 310 : Weight = 3.191
Node 173 -- Node 179 : Weight = 1.759
Node 173 -- Node 203 : Weight = 1.423
Node 173 -- Node 225 : Weight = 1.786
Node 173 -- Node 226 : Weight = 1.524
Node 174 -- Node 201 : Weight = 1.643
Node 175 -- Node 196 : Weight = 1.859
Node 176 -- Node 251 : Weight = 1.548
Node 176 -- Node 252 : Weight = 1.771

Node 177 -- Node 183 : Weight = 2.270
Node 179 -- Node 340 : Weight = 1.825
Node 180 -- Node 188 : Weight = 1.270
Node 181 -- Node 241 : Weight = 2.409
Node 182 -- Node 221 : Weight = 1.728
Node 182 -- Node 242 : Weight = 2.149
Node 182 -- Node 350 : Weight = 2.818
Node 183 -- Node 207 : Weight = 2.153
Node 183 -- Node 246 : Weight = 1.797
Node 184 -- Node 249 : Weight = 2.129
Node 185 -- Node 211 : Weight = 2.300
Node 185 -- Node 289 : Weight = 1.886
Node 185 -- Node 322 : Weight = 2.605
Node 186 -- Node 297 : Weight = 1.519
Node 188 -- Node 298 : Weight = 1.753
Node 188 -- Node 342 : Weight = 2.205
Node 189 -- Node 232 : Weight = 1.960
Node 191 -- Node 317 : Weight = 2.637
Node 192 -- Node 251 : Weight = 1.791
Node 193 -- Node 197 : Weight = 2.936
Node 196 -- Node 257 : Weight = 1.533
Node 198 -- Node 231 : Weight = 2.156
Node 198 -- Node 321 : Weight = 2.330
Node 199 -- Node 252 : Weight = 1.467
Node 200 -- Node 257 : Weight = 1.627
Node 200 -- Node 301 : Weight = 1.683
Node 200 -- Node 339 : Weight = 1.551
Node 201 -- Node 221 : Weight = 1.818
Node 201 -- Node 236 : Weight = 1.635
Node 202 -- Node 302 : Weight = 1.854
Node 203 -- Node 268 : Weight = 1.493
Node 203 -- Node 329 : Weight = 1.682
Node 204 -- Node 255 : Weight = 1.807
Node 204 -- Node 275 : Weight = 1.391
Node 204 -- Node 304 : Weight = 1.527
Node 205 -- Node 313 : Weight = 1.457
Node 206 -- Node 263 : Weight = 1.669
Node 206 -- Node 268 : Weight = 1.424
Node 208 -- Node 295 : Weight = 4.351
Node 209 -- Node 271 : Weight = 1.899
Node 209 -- Node 274 : Weight = 1.913
Node 210 -- Node 286 : Weight = 2.060
Node 213 -- Node 265 : Weight = 1.844
Node 214 -- Node 249 : Weight = 2.252
Node 214 -- Node 285 : Weight = 2.506
Node 215 -- Node 283 : Weight = 1.696
Node 215 -- Node 286 : Weight = 1.473
Node 216 -- Node 224 : Weight = 2.357
Node 216 -- Node 312 : Weight = 2.222
Node 217 -- Node 288 : Weight = 1.556
Node 221 -- Node 315 : Weight = 1.807
Node 222 -- Node 257 : Weight = 1.613
Node 223 -- Node 320 : Weight = 2.221
Node 226 -- Node 243 : Weight = 1.918
Node 229 -- Node 304 : Weight = 1.585
Node 230 -- Node 252 : Weight = 2.013
Node 230 -- Node 300 : Weight = 2.484
Node 231 -- Node 327 : Weight = 1.767
Node 231 -- Node 330 : Weight = 2.379
Node 232 -- Node 254 : Weight = 1.787
Node 232 -- Node 311 : Weight = 2.094
Node 232 -- Node 314 : Weight = 1.651
Node 233 -- Node 265 : Weight = 2.816
Node 235 -- Node 298 : Weight = 2.121
Node 237 -- Node 342 : Weight = 2.664

Node 238 -- Node 246 : Weight = 2.123
Node 239 -- Node 315 : Weight = 1.809
Node 243 -- Node 250 : Weight = 1.595
Node 243 -- Node 281 : Weight = 1.794
Node 243 -- Node 345 : Weight = 2.597
Node 244 -- Node 295 : Weight = 4.320
Node 246 -- Node 278 : Weight = 2.155
Node 248 -- Node 319 : Weight = 1.291
Node 249 -- Node 253 : Weight = 2.263
Node 249 -- Node 289 : Weight = 2.207
Node 250 -- Node 343 : Weight = 2.319
Node 251 -- Node 319 : Weight = 1.846
Node 253 -- Node 280 : Weight = 3.244
Node 253 -- Node 334 : Weight = 2.173
Node 254 -- Node 256 : Weight = 1.668
Node 254 -- Node 313 : Weight = 1.479
Node 258 -- Node 313 : Weight = 1.404
Node 259 -- Node 278 : Weight = 2.788
Node 262 -- Node 320 : Weight = 2.414
Node 262 -- Node 354 : Weight = 2.530
Node 263 -- Node 548 : Weight = 3.593
Node 264 -- Node 287 : Weight = 2.002
Node 264 -- Node 296 : Weight = 1.870
Node 266 -- Node 270 : Weight = 2.752
Node 266 -- Node 273 : Weight = 2.220
Node 266 -- Node 348 : Weight = 1.615
Node 266 -- Node 349 : Weight = 2.890
Node 267 -- Node 314 : Weight = 2.130
Node 268 -- Node 304 : Weight = 1.518
Node 268 -- Node 324 : Weight = 1.697
Node 269 -- Node 337 : Weight = 2.106
Node 271 -- Node 272 : Weight = 1.028
Node 271 -- Node 298 : Weight = 1.982
Node 271 -- Node 337 : Weight = 1.739
Node 277 -- Node 335 : Weight = 3.688
Node 279 -- Node 332 : Weight = 3.529
Node 279 -- Node 342 : Weight = 2.234
Node 280 -- Node 331 : Weight = 2.801
Node 282 -- Node 343 : Weight = 2.885
Node 284 -- Node 351 : Weight = 2.789
Node 284 -- Node 355 : Weight = 3.401
Node 286 -- Node 304 : Weight = 1.735
Node 287 -- Node 352 : Weight = 2.324
Node 291 -- Node 296 : Weight = 1.631
Node 291 -- Node 393 : Weight = 1.952
Node 292 -- Node 409 : Weight = 2.179
Node 293 -- Node 329 : Weight = 1.782
Node 297 -- Node 306 : Weight = 1.543
Node 297 -- Node 317 : Weight = 1.662
Node 302 -- Node 339 : Weight = 1.951
Node 303 -- Node 323 : Weight = 2.247
Node 303 -- Node 409 : Weight = 2.080
Node 305 -- Node 409 : Weight = 2.388
Node 306 -- Node 315 : Weight = 1.735
Node 307 -- Node 308 : Weight = 5.361
Node 308 -- Node 310 : Weight = 5.761
Node 309 -- Node 317 : Weight = 2.348
Node 310 -- Node 318 : Weight = 4.338
Node 316 -- Node 320 : Weight = 2.409
Node 320 -- Node 338 : Weight = 2.047
Node 320 -- Node 421 : Weight = 2.360
Node 321 -- Node 341 : Weight = 2.241
Node 323 -- Node 328 : Weight = 2.022
Node 323 -- Node 397 : Weight = 1.998
Node 327 -- Node 502 : Weight = 2.668

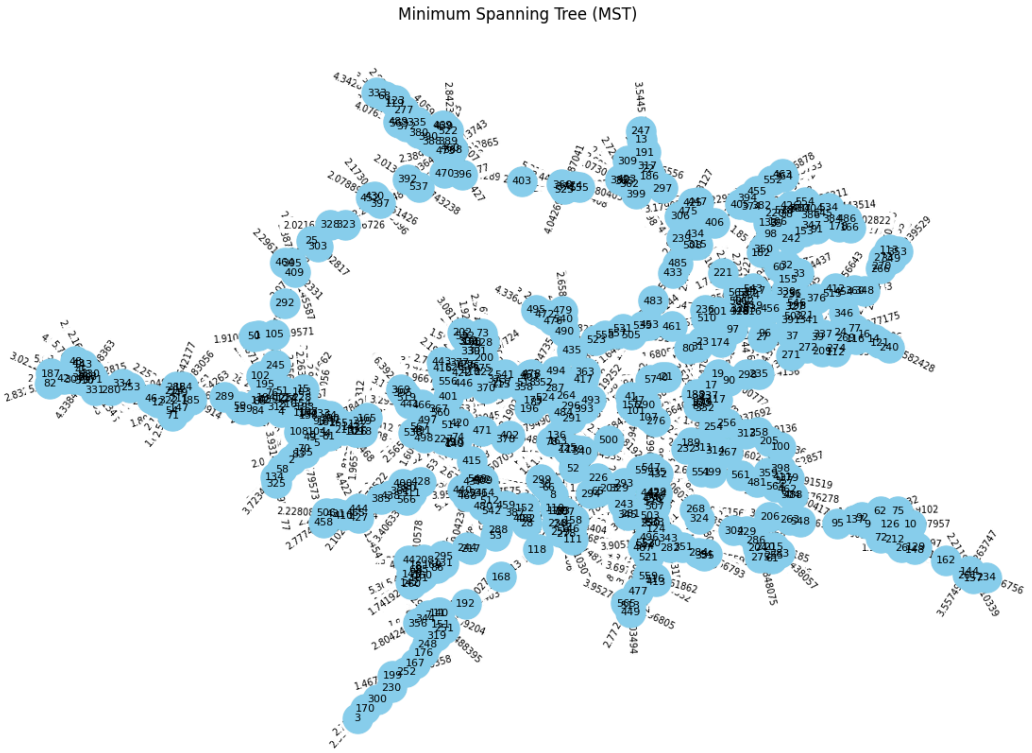
Node 335 -- Node 390 : Weight = 2.155
Node 336 -- Node 339 : Weight = 1.923
Node 336 -- Node 532 : Weight = 3.081
Node 337 -- Node 346 : Weight = 1.855
Node 339 -- Node 436 : Weight = 2.059
Node 343 -- Node 351 : Weight = 3.370
Node 344 -- Node 356 : Weight = 2.804
Node 346 -- Node 348 : Weight = 1.633
Node 349 -- Node 353 : Weight = 2.893
Node 357 -- Node 411 : Weight = 4.841
Node 358 -- Node 518 : Weight = 2.486
Node 359 -- Node 398 : Weight = 2.687
Node 359 -- Node 411 : Weight = 3.719
Node 359 -- Node 561 : Weight = 2.743
Node 360 -- Node 454 : Weight = 7.892
Node 361 -- Node 550 : Weight = 3.104
Node 362 -- Node 365 : Weight = 2.695
Node 362 -- Node 385 : Weight = 3.073
Node 362 -- Node 399 : Weight = 2.513
Node 362 -- Node 423 : Weight = 2.729
Node 363 -- Node 417 : Weight = 2.792
Node 363 -- Node 435 : Weight = 1.868
Node 363 -- Node 484 : Weight = 2.029
Node 363 -- Node 494 : Weight = 1.650
Node 363 -- Node 505 : Weight = 1.400
Node 364 -- Node 552 : Weight = 2.653
Node 366 -- Node 371 : Weight = 5.318
Node 367 -- Node 401 : Weight = 2.129
Node 367 -- Node 497 : Weight = 2.031
Node 368 -- Node 473 : Weight = 2.390
Node 369 -- Node 395 : Weight = 6.971
Node 370 -- Node 418 : Weight = 2.142
Node 371 -- Node 474 : Weight = 3.027
Node 371 -- Node 525 : Weight = 4.043
Node 372 -- Node 380 : Weight = 2.995
Node 372 -- Node 489 : Weight = 3.952
Node 372 -- Node 563 : Weight = 4.076
Node 373 -- Node 446 : Weight = 1.595
Node 373 -- Node 451 : Weight = 1.927
Node 374 -- Node 382 : Weight = 3.119
Node 374 -- Node 425 : Weight = 2.138
Node 374 -- Node 426 : Weight = 2.493
Node 374 -- Node 528 : Weight = 2.537
Node 375 -- Node 377 : Weight = 2.526
Node 375 -- Node 541 : Weight = 2.442
Node 376 -- Node 515 : Weight = 4.349
Node 376 -- Node 546 : Weight = 3.989
Node 377 -- Node 443 : Weight = 3.357
Node 378 -- Node 402 : Weight = 2.351
Node 379 -- Node 411 : Weight = 4.360
Node 380 -- Node 388 : Weight = 2.801
Node 381 -- Node 428 : Weight = 2.653
Node 383 -- Node 438 : Weight = 2.147
Node 384 -- Node 386 : Weight = 2.708
Node 384 -- Node 486 : Weight = 3.193
Node 386 -- Node 528 : Weight = 2.776
Node 387 -- Node 408 : Weight = 2.893
Node 387 -- Node 459 : Weight = 3.004
Node 388 -- Node 473 : Weight = 2.222
Node 389 -- Node 473 : Weight = 2.943
Node 389 -- Node 522 : Weight = 2.842
Node 390 -- Node 473 : Weight = 2.155
Node 391 -- Node 502 : Weight = 4.576
Node 392 -- Node 537 : Weight = 2.013
Node 393 -- Node 493 : Weight = 1.985

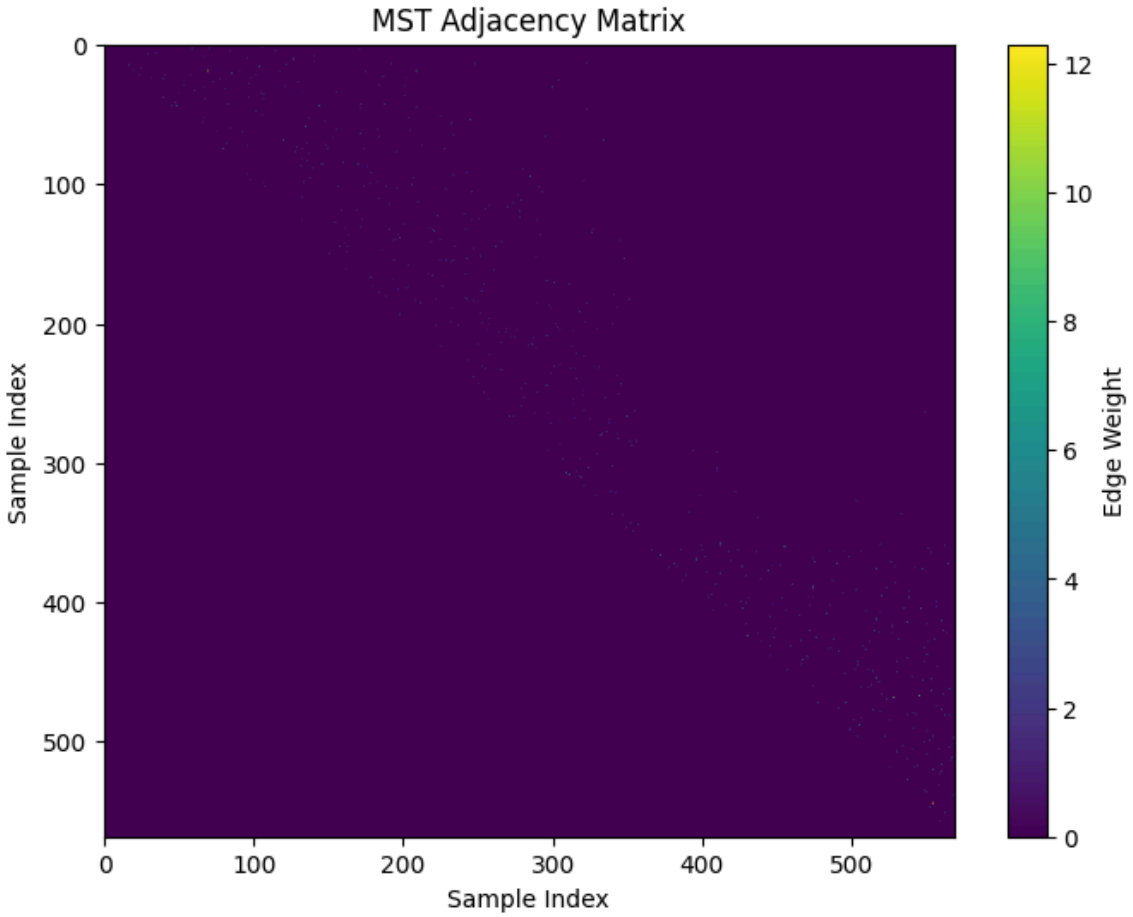
Node 394 -- Node 405 : Weight = 2.637
Node 395 -- Node 519 : Weight = 6.392
Node 396 -- Node 403 : Weight = 1.972
Node 396 -- Node 470 : Weight = 2.152
Node 396 -- Node 473 : Weight = 1.842
Node 396 -- Node 537 : Weight = 2.262
Node 397 -- Node 430 : Weight = 2.173
Node 397 -- Node 452 : Weight = 2.079
Node 397 -- Node 537 : Weight = 2.244
Node 399 -- Node 425 : Weight = 2.245
Node 399 -- Node 555 : Weight = 2.118
Node 400 -- Node 428 : Weight = 2.816
Node 401 -- Node 446 : Weight = 1.807
Node 402 -- Node 415 : Weight = 2.553
Node 402 -- Node 471 : Weight = 2.580
Node 402 -- Node 484 : Weight = 2.213
Node 403 -- Node 555 : Weight = 2.155
Node 404 -- Node 426 : Weight = 3.167
Node 404 -- Node 534 : Weight = 3.449
Node 404 -- Node 545 : Weight = 4.517
Node 405 -- Node 406 : Weight = 1.996
Node 405 -- Node 455 : Weight = 2.642
Node 406 -- Node 508 : Weight = 1.976
Node 407 -- Node 518 : Weight = 1.803
Node 408 -- Node 482 : Weight = 3.889
Node 409 -- Node 464 : Weight = 2.296
Node 410 -- Node 444 : Weight = 2.735
Node 412 -- Node 515 : Weight = 7.117
Node 413 -- Node 559 : Weight = 4.354
Node 414 -- Node 536 : Weight = 5.155
Node 415 -- Node 428 : Weight = 1.955
Node 415 -- Node 549 : Weight = 2.061
Node 416 -- Node 526 : Weight = 2.922
Node 418 -- Node 526 : Weight = 2.783
Node 419 -- Node 448 : Weight = 2.151
Node 420 -- Node 471 : Weight = 2.607
Node 420 -- Node 514 : Weight = 2.976
Node 420 -- Node 560 : Weight = 2.472
Node 421 -- Node 517 : Weight = 3.320
Node 421 -- Node 562 : Weight = 2.061
Node 422 -- Node 436 : Weight = 2.273
Node 422 -- Node 446 : Weight = 1.969
Node 422 -- Node 526 : Weight = 2.191
Node 424 -- Node 429 : Weight = 8.078
Node 424 -- Node 568 : Weight = 5.942
Node 425 -- Node 447 : Weight = 3.179
Node 425 -- Node 475 : Weight = 2.632
Node 426 -- Node 487 : Weight = 3.842
Node 426 -- Node 530 : Weight = 3.179
Node 426 -- Node 554 : Weight = 3.420
Node 427 -- Node 465 : Weight = 4.423
Node 428 -- Node 438 : Weight = 2.521
Node 428 -- Node 511 : Weight = 2.753
Node 428 -- Node 550 : Weight = 2.168
Node 431 -- Node 444 : Weight = 2.250
Node 431 -- Node 458 : Weight = 2.777
Node 431 -- Node 506 : Weight = 2.228
Node 432 -- Node 547 : Weight = 2.743
Node 433 -- Node 483 : Weight = 1.983
Node 433 -- Node 485 : Weight = 2.346
Node 433 -- Node 508 : Weight = 1.808
Node 434 -- Node 508 : Weight = 2.443
Node 435 -- Node 490 : Weight = 2.760
Node 437 -- Node 549 : Weight = 5.199
Node 438 -- Node 444 : Weight = 1.907

Node 439 -- Node 469 : Weight = 4.849
Node 440 -- Node 536 : Weight = 2.555
Node 441 -- Node 466 : Weight = 3.293
Node 442 -- Node 503 : Weight = 3.090
Node 442 -- Node 529 : Weight = 2.834
Node 442 -- Node 547 : Weight = 2.433
Node 444 -- Node 465 : Weight = 2.103
Node 445 -- Node 503 : Weight = 3.233
Node 446 -- Node 556 : Weight = 1.967
Node 448 -- Node 456 : Weight = 2.416
Node 448 -- Node 501 : Weight = 2.222
Node 448 -- Node 510 : Weight = 2.499
Node 449 -- Node 513 : Weight = 3.724
Node 450 -- Node 529 : Weight = 3.436
Node 450 -- Node 568 : Weight = 3.573
Node 451 -- Node 494 : Weight = 2.399
Node 453 -- Node 539 : Weight = 2.110
Node 454 -- Node 515 : Weight = 7.072
Node 455 -- Node 552 : Weight = 2.811
Node 456 -- Node 546 : Weight = 2.801
Node 457 -- Node 517 : Weight = 2.985
Node 459 -- Node 512 : Weight = 2.877
Node 460 -- Node 473 : Weight = 2.036
Node 461 -- Node 505 : Weight = 1.888
Node 461 -- Node 510 : Weight = 2.451
Node 462 -- Node 564 : Weight = 3.851
Node 463 -- Node 552 : Weight = 3.908
Node 466 -- Node 519 : Weight = 3.127
Node 466 -- Node 560 : Weight = 3.105
Node 467 -- Node 544 : Weight = 8.282
Node 468 -- Node 527 : Weight = 9.042
Node 469 -- Node 522 : Weight = 4.059
Node 472 -- Node 476 : Weight = 3.100
Node 472 -- Node 495 : Weight = 4.336
Node 474 -- Node 555 : Weight = 2.449
Node 476 -- Node 490 : Weight = 2.629
Node 477 -- Node 513 : Weight = 2.779
Node 477 -- Node 559 : Weight = 2.911
Node 478 -- Node 518 : Weight = 2.190
Node 479 -- Node 540 : Weight = 3.452
Node 480 -- Node 536 : Weight = 2.247
Node 480 -- Node 542 : Weight = 3.019
Node 481 -- Node 561 : Weight = 3.515
Node 481 -- Node 564 : Weight = 3.520
Node 483 -- Node 505 : Weight = 1.426
Node 484 -- Node 500 : Weight = 2.026
Node 484 -- Node 524 : Weight = 2.196
Node 488 -- Node 504 : Weight = 5.248
Node 490 -- Node 540 : Weight = 2.658
Node 491 -- Node 497 : Weight = 1.608
Node 491 -- Node 498 : Weight = 2.804
Node 492 -- Node 549 : Weight = 2.851
Node 496 -- Node 503 : Weight = 3.112
Node 496 -- Node 521 : Weight = 3.050
Node 496 -- Node 553 : Weight = 3.658
Node 497 -- Node 567 : Weight = 3.103
Node 499 -- Node 551 : Weight = 2.855
Node 500 -- Node 547 : Weight = 2.170
Node 503 -- Node 533 : Weight = 3.584
Node 504 -- Node 564 : Weight = 3.237
Node 505 -- Node 539 : Weight = 1.646
Node 505 -- Node 557 : Weight = 1.821
Node 507 -- Node 529 : Weight = 3.814
Node 509 -- Node 549 : Weight = 3.853
Node 511 -- Node 566 : Weight = 3.406

```
Node 512 -- Node 536 : Weight = 2.670
Node 513 -- Node 565 : Weight = 3.953
Node 516 -- Node 559 : Weight = 3.698
Node 517 -- Node 543 : Weight = 3.183
Node 518 -- Node 524 : Weight = 2.164
Node 518 -- Node 541 : Weight = 2.515
Node 520 -- Node 553 : Weight = 3.905
Node 521 -- Node 559 : Weight = 2.462
Node 523 -- Node 558 : Weight = 2.862
Node 527 -- Node 536 : Weight = 3.951
Node 531 -- Node 557 : Weight = 1.808
Node 535 -- Node 547 : Weight = 2.819
Node 536 -- Node 549 : Weight = 2.543
Node 538 -- Node 567 : Weight = 2.566
Node 544 -- Node 553 : Weight = 12.300
Node 547 -- Node 551 : Weight = 2.034
Node 551 -- Node 561 : Weight = 2.618
Node 557 -- Node 558 : Weight = 1.949

Number of MST Nodes: 569
Number of MST Edges: 568
```





Task 6: Implement Graph-Based Clustering ($K = 2$)

- Sort the edges of the MST in descending order by edge weight.
- Remove the largest $K - 1$ edge (1 largest edge for 2 clusters) to split the MST into 2 connected components.
- Assign final cluster labels based on the resulting component indices.

```
# Task 6: Graph-Based Clustering (K = 2)

import networkx as nx
import matplotlib.pyplot as plt

# Make a copy of the MST
cluster_graph = MST.copy()

# Get all MST edges with their weights
edges = list(cluster_graph.edges(data=True))

# Sort edges by weight in descending order
edges_sorted = sorted(
    edges,
    key=lambda x: x[2]["weight"],
    reverse=True
)
```

```python
# Display the largest edge
largest_edge = edges_sorted[0]

print("Largest MST Edge:")
print(
    f"Node {largest_edge[0]} -- Node {largest_edge[1]} "
    f": Weight = {largest_edge[2]['weight']:.3f}"
)

# Remove the largest edge
cluster_graph.remove_edge(
    largest_edge[0],
    largest_edge[1]
)

# Find the connected components
components = list(nx.connected_components(cluster_graph))

print("\nNumber of Clusters:", len(components))

# Assign cluster labels
cluster_labels = np.zeros(len(X_scaled), dtype=int)

for cluster_id, component in enumerate(components):
    for node in component:
        cluster_labels[node] = cluster_id

# Display cluster sizes
print("\nCluster Sizes:")
for cluster_id in range(len(components)):
    print(
        f"Cluster {cluster_id}: "
        f"{np.sum(cluster_labels == cluster_id)} samples"
    )

# Display first 20 cluster labels
print("\nFirst 20 Cluster Labels:")
print(cluster_labels[:20])

# ----------------------------
# Visualize the 2 Clusters
# ----------------------------

plt.figure(figsize=(12, 8))

pos = nx.spring_layout(cluster_graph, seed=42)

nx.draw(
    cluster_graph,
    pos,
    node_color=cluster_labels,
    cmap="viridis",
    with_labels=False,
    node_size=40,
    edge_color="gray"
)
```

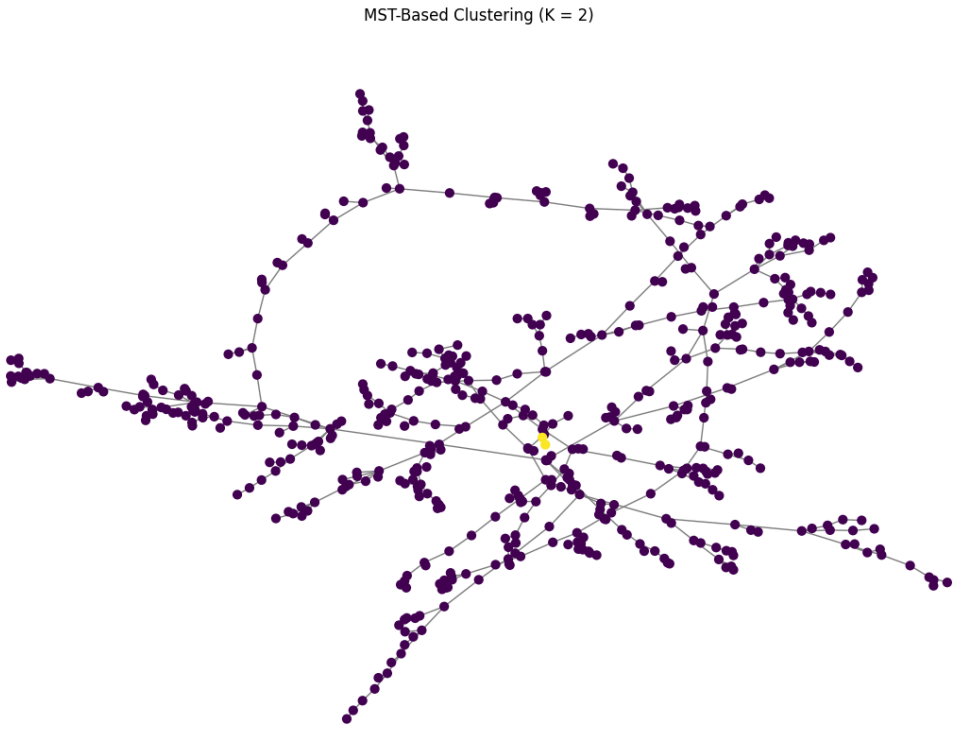
```
plt.title("MST-Based Clustering (K = 2)")
plt.show()

Largest MST Edge:
Node 544 -- Node 553 : Weight = 12.300

Number of Clusters: 2

Cluster Sizes:
Cluster 0: 567 samples
Cluster 1: 2 samples

First 20 Cluster Labels:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```



Task 7: valuate MST Clustering Performance

Evaluate cluster assignment against ground truth labels using:

- Silhouette Score
- Adjusted Rand Index (ARI)
- Normalized Mutual Information (NMI)

```
# Task 7: Evaluate MST Clustering Performance

from sklearn.metrics import silhouette_score, adjusted_rand_score,
        normalized_mutual_info_score
```

```python
# Silhouette Score
sil_score = silhouette_score(X_scaled, cluster_labels)

print("Silhouette Score:", round(sil_score, 4))

# Find the diagnosis/ground-truth column automatically
target_col = next(
    (col for col in df.columns if col.strip().lower() == "diagnosis"),
    None
)

if target_col is not None:

    # Convert ground truth labels to numbers
    y_true = df[target_col].astype(str).str.strip().str.upper()

    # Convert M/B labels to numbers
    y_true = y_true.map({"M": 1, "B": 0})

    # ARI
    ari_score = adjusted_rand_score(y_true, cluster_labels)

    # NMI
    nmi_score = normalized_mutual_info_score(y_true, cluster_labels)

    print("Adjusted Rand Index (ARI):", round(ari_score, 4))
    print("Normalized Mutual Information (NMI):", round(nmi_score, 4))

else:
    print("Diagnosis column not found.")
    print("Available columns are:")
    print(df.columns.tolist())
```

```
Silhouette Score: 0.6547
Diagnosis column not found.
Available columns are:
['Unnamed: 0', 'x.radius_mean', 'x.texture_mean', 'x.perimeter_mean',
'x.area_mean', 'x.smoothness_mean', 'x.compactness_mean', 'x.concavity_mean',
'x.concave_pts_mean', 'x.symmetry_mean', 'x.fractal_dim_mean', 'x.radius_se',
'x.texture_se', 'x.perimeter_se', 'x.area_se', 'x.smoothness_se',
'x.compactness_se', 'x.concavity_se', 'x.concave_pts_se', 'x.symmetry_se',
'x.fractal_dim_se', 'x.radius_worst', 'x.texture_worst', 'x.perimeter_worst',
'x.area_worst', 'x.smoothness_worst', 'x.compactness_worst',
'x.concavity_worst', 'x.concave_pts_worst', 'x.symmetry_worst',
'x.fractal_dim_worst', 'y']
```

Task 8: Visualize MST and Graph-Based Clusters

- Select two representative features (e.g., mean radius vs. mean texture) for 2D plotting.
- Plot 1: Unclustered standardized data points.
- Plot 2: Minimum Spanning Tree (MST) edge connections overlaid on the data.
- Plot 3: Final clusters formed after cutting the $K - 1$ largest edge(s).

```python
# ============================================================
# TASK 8: VISUALIZE MST AND GRAPH-BASED CLUSTERS
# ============================================================

import numpy as np
import matplotlib.pyplot as plt

# ------------------------------------------------------------
# STEP 1: Select two representative features
# Mean Radius and Mean Texture
# ------------------------------------------------------------

# Find Mean Radius column
radius_col = next(
    (col for col in X.columns
     if "radius" in col.lower() and "mean" in col.lower()),
    X.columns[0]
)

# Find Mean Texture column
texture_col = next(
    (col for col in X.columns
     if "texture" in col.lower() and "mean" in col.lower()),
    X.columns[1]
)

# Get column positions
radius_index = list(X.columns).index(radius_col)
texture_index = list(X.columns).index(texture_col)

# Get standardized values
x_axis = X_scaled.iloc[:, radius_index]
y_axis = X_scaled.iloc[:, texture_index]

print("Feature 1:", radius_col)
print("Feature 2:", texture_col)


# ============================================================
# PLOT 1: UNCLUSTERED STANDARDIZED DATA
# ============================================================

plt.figure(figsize=(8, 6))

plt.scatter(
    x_axis,
    y_axis,
    s=40,
    alpha=0.7
)

plt.xlabel("Mean Radius (Standardized)")
plt.ylabel("Mean Texture (Standardized)")
plt.title("Unclustered Standardized Data")

plt.grid(True, alpha=0.3)
plt.show()
```

```
# ============================================================
# PLOT 2: MST EDGE CONNECTIONS
# ============================================================

plt.figure(figsize=(10, 7))

# Plot all data points
plt.scatter(
    x_axis,
    y_axis,
    s=25,
    alpha=0.7
)

# Draw MST edges
for u, v, data in MST.edges(data=True):

    plt.plot(
        [x_axis.iloc[u], x_axis.iloc[v]],
        [y_axis.iloc[u], y_axis.iloc[v]],
        linewidth=0.5,
        alpha=0.5
    )

plt.xlabel("Mean Radius (Standardized)")
plt.ylabel("Mean Texture (Standardized)")
plt.title("Minimum Spanning Tree (MST)")

plt.grid(True, alpha=0.3)
plt.show()


# ============================================================
# PLOT 3: FINAL GRAPH-BASED CLUSTERS
# ============================================================

plt.figure(figsize=(8, 6))

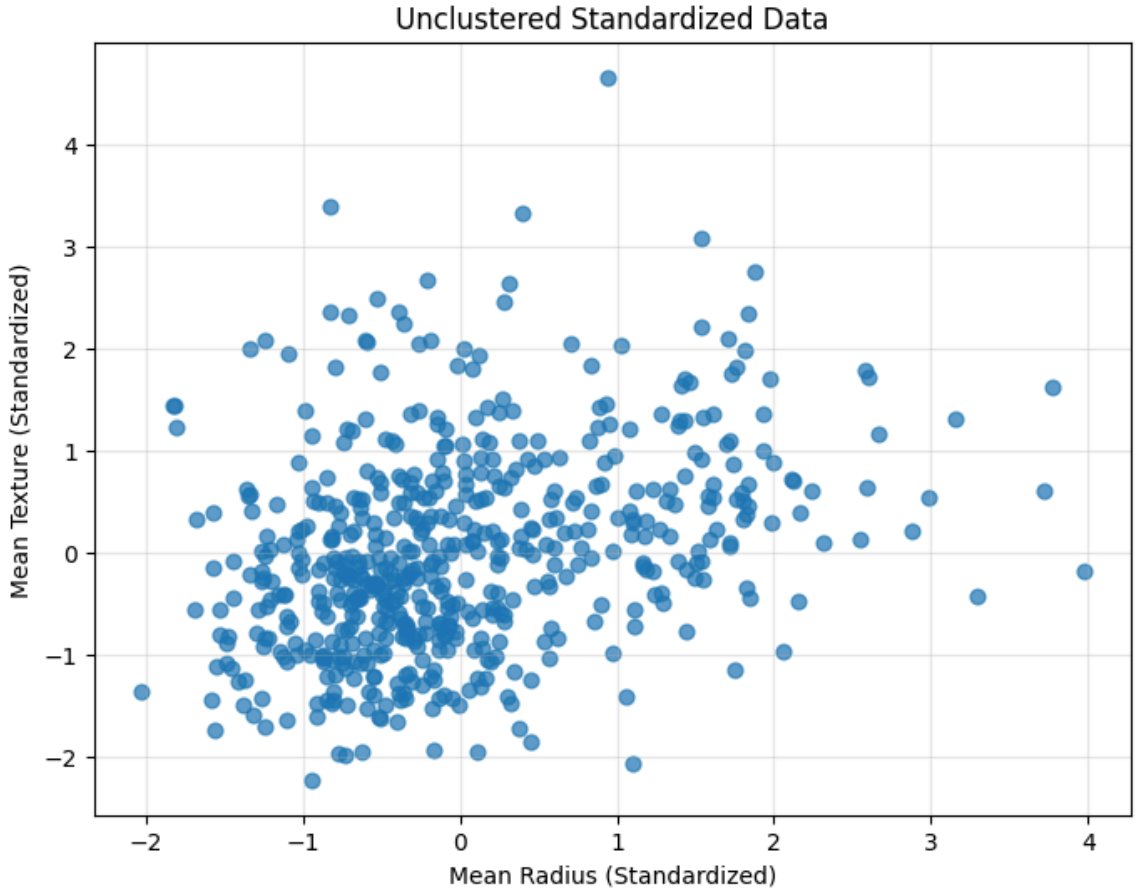
plt.scatter(
    x_axis,
    y_axis,
    c=cluster_labels,
    cmap="viridis",
    s=40,
    alpha=0.8
)

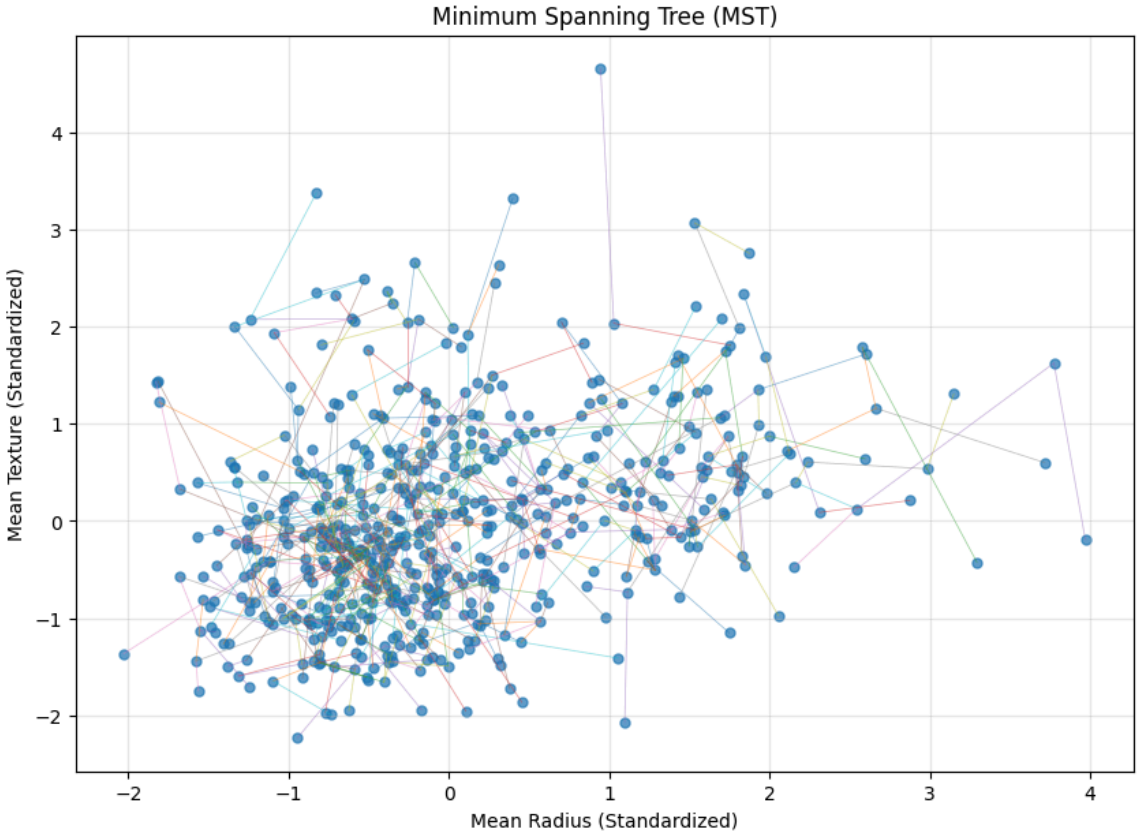
plt.xlabel("Mean Radius (Standardized)")
plt.ylabel("Mean Texture (Standardized)")
plt.title("Final Graph-Based Clusters (K = 2)")

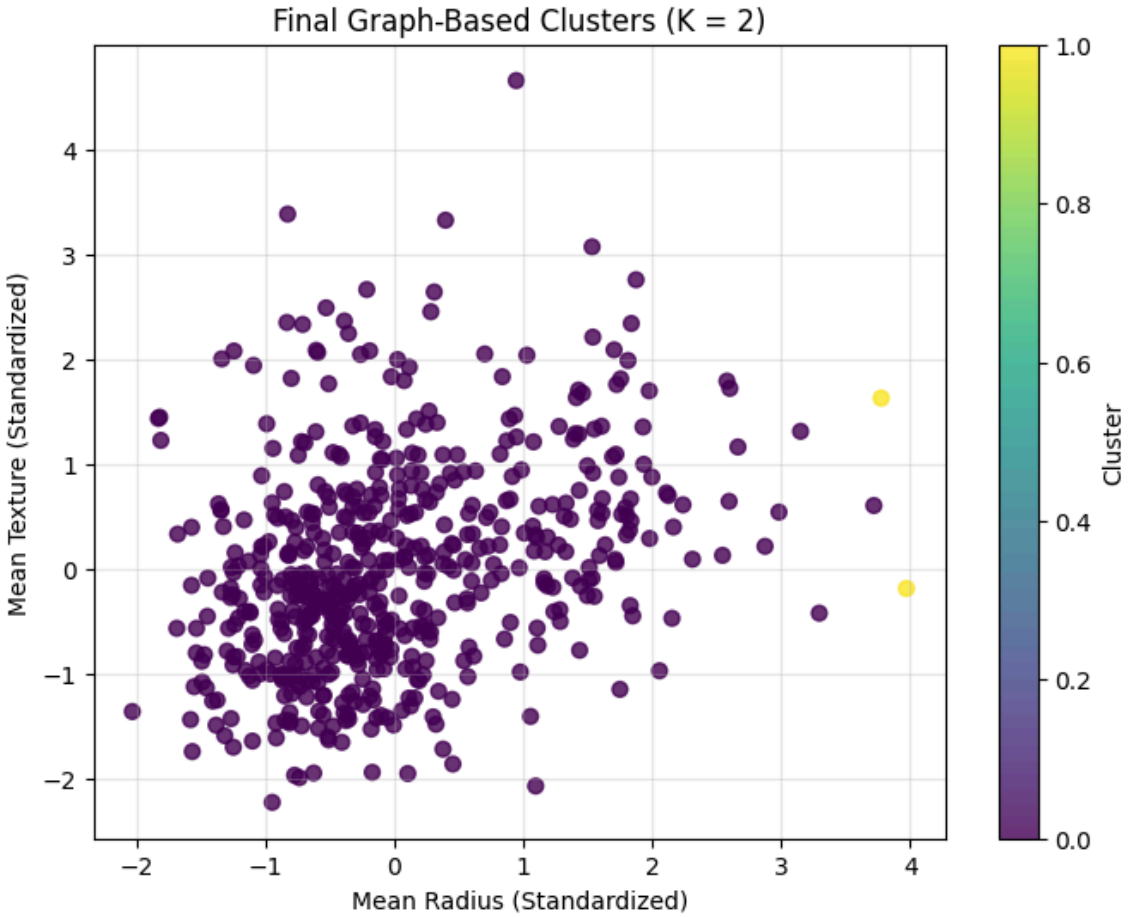
plt.colorbar(label="Cluster")

plt.grid(True, alpha=0.3)
plt.show()

Feature 1: x.radius_mean
Feature 2: x.texture_mean
```


Unclustered Standardized Data
Mean Texture (Standardized)
Mean Radius (Standardized)
4
3
2
1
0
−1
−2
−2
−1
0
1
2
3
4


Minimum Spanning Tree (MST)
Mean Texture (Standardized)
Mean Radius (Standardized)
4
3
2
1
0
−1
−2
−2
−1
0
1
2
3
4



Conclusion

The experiment successfully demonstrated the implementation of Graph-Based Clustering using a Minimum Spanning Tree (MST) on the Breast Cancer Wisconsin (Diagnostic) dataset. The dataset was preprocessed, cleaned, and standardized before computing pairwise distances. The MST was generated, and by removing the largest edge(s), clusters representing tumor categories were obtained, evaluated using clustering metrics, and visualized.